DakshSetu

AI-Assisted Skill Assessment for Recognition of Prior Learning

Technical Report

Programme	Smart India Hackathon 2026
Problem statement	SIH26242 — AI-Assisted Skill Assessment Tool for Recognition of Prior Learning (RPL)
Product	DakshSetu (worker PWA + assessor portal + certification)
Repository	`sih26242-rpl` — npm-workspaces monorepo
Deliverable	Web platform: contract-first API, offline-first PWA, AI-assisted scoring, verifiable certification
Version	1.0
Date	2026

This report documents the architecture, domain model, assessment pipeline, security model, testing evidence, and operations of the DakshSetu platform. All figures are taken from the working repository.

Executive summary

Recognition of Prior Learning (RPL) lets experienced workers obtain formal NSQF-aligned certification for skills already acquired on the job, without retraining from scratch. The bottleneck is assessment: a fair, consistent, auditable and low-cost way to judge a worker's competence. DakshSetu is an end-to-end platform that runs a worker from self-declaration, through AI-assisted trade mapping and a personalised practical plan, to evidence capture, AI-assisted scoring, human assessment, and an instantly verifiable digital certificate.

The system is built contract-first: a single TypeScript package (`@sih/contracts`) defines every Zod schema, the assessment state machine, and the complete HTTP route table (43 endpoints) from which the OpenAPI document, the backend request validation, and the frontend typed client are all derived. The backend is a layered Express 5 / MongoDB service with pluggable drivers for AI, storage, upload and queueing; the frontend is a React 18 PWA that works offline. The scoring engine is *deterministic-first*: objective items are auto-scored, AI is used only where it adds value, and a human assessor always retains precedence. Critical safety competencies act as a veto. Certificates carry a QR-linked public verifier and a SHA-256 evidence/audit hash chain. The backend is covered by 124 automated tests across 14 suites; the full lifecycle was additionally exercised end-to-end through the UI.

Contents

Executive summary 1

1 Introduction 3
- 1.1 Context . . . 3
- 1.2 Problem statement (SIH26242) . . . 3
- 1.3 Design principles . . . 3

2 Solution overview 3

3 Architecture 4
- 3.1 Layers . . . 4
- 3.2 Runtime topology . . . 4

4 Contract-first design 4
- 4.1 Entities and schemas (Zod) . . . 4
- 4.2 The route table . . . 4
- 4.3 State machine as a contract . . . 4

5 Domain model and lifecycle 5
- 5.1 Roles . . . 5
- 5.2 Entities . . . 5
- 5.3 Task types . . . 5

6 Assessment pipeline 5
- 6.1 Declaration and mapping . . . 5
- 6.2 Plan generation . . . 5
- 6.3 Task runner and evidence capture . . . 6
- 6.4 AI analysis . . . 6
- 6.5 Scoring engine . . . 6
- 6.6 Review and decision . . . 6
- 6.7 Report, certificate and verification . . . 6

7 Offline-first and synchronisation 6

8 Security, privacy and integrity 7

9 Fairness and multilingual support **7**
9.1 Fairness . . . 7
9.2 Multilingual . . . 7

10 Frontend — DakshSetu PWA **7**
10.1 Stack . . . 7
10.2 Portals . . . 7
10.3 Realtime and resilience . . . 8

11 Testing and verification **8**
11.1 Automated suite . . . 8
11.2 End-to-end walkthrough . . . 8
11.3 Continuous checks . . . 8

12 Operations and deployment **8**
12.1 Local development . . . 8
12.2 Containers . . . 8
12.3 Configuration . . . 8

13 Trade pack catalogue **9**

14 API surface **9**

15 Limitations and future work **9**

A API route reference **10**

B State machine and task types **11**
B.1 Statuses . . . 11
B.2 Allowed transitions . . . 11

1 Introduction

1.1 Context

Skill India's RPL track certifies workers against National Skills Qualification Framework (NSQF) Qualification Packs (QPs) defined by sector skill councils. A worker with ten years of site experience may have no paper credential; conversely, a certificate should mean the holder can actually do the job. Traditional assessor-led RPL is expensive, geographically constrained, and can vary between assessors. Digital assessment can scale it, but naive automation risks unfair or unexplainable outcomes — particularly on safety-critical competencies.

1.2 Problem statement (SIH26242)

Build an AI-assisted skill assessment tool for RPL that:

- lets workers self-declare experience, then maps them to the most suitable NSQF Qualification Pack;
- generates a personalised, evidence-based assessment plan;
- captures practical evidence (photos, video, narration) with integrity signals;
- uses AI to assist scoring while keeping decisions explainable, fair and human-controlled;
- issues a tamper-evident, publicly verifiable certificate.

1.3 Design principles

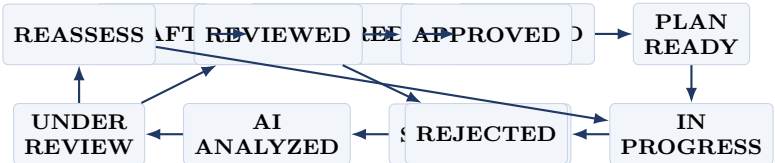
1. **Contract-first.** One source of truth for schemas, routes and the state machine; generated OpenAPI; no hand-written duplicate types.
2. **Deterministic-first AI.** Objective questions are auto-scored exactly; AI augments subjective items and never silently overrides a human.
3. **Safety as a veto.** Missing a critical-safety competency cannot be averaged away.
4. **Offline-first.** Low-connectivity worksites are the norm; the PWA queues work and reconciles later.
5. **Auditable by construction.** Every transition is recorded on a hash-chained audit log; evidence bytes are hashed.
6. **Trilingual.** English, Hindi and Marathi across the UI, tasks and narration.

2 Solution overview

DakshSetu is a monorepo of three npm workspaces:

Workspace	Responsibility
`@sih/contracts`	Zod entities and API schemas, the assessment state machine, route table, trade-pack schema, generated OpenAPI.
`@sih/backend`	Express 5 API, MongoDB persistence, auth, evidence storage (tus), AI/vision/ASR adapters, scoring, review, certification, queue worker, realtime.
`@sih/frontend`	**DakshSetu** React PWA: worker portal, assessor portal, public verifier.

A worker's journey is a linear, resumable state machine that can loop once for reassessment:



3 Architecture

3.1 Layers

The backend follows a strict layering so that business logic is testable without HTTP and HTTP concerns never leak into domain code:

- **Routes / handlers** (`src/routes/handlers`) — thin; parse with contract schemas, call a service, shape the response.
- **Services** (`src/services`) — domain logic: assessment lifecycle, mapping, planning, analysis, scoring, review, certification, fairness, search, sync.
- **Models** (`src/models`) — Mongoose documents mirroring the contract entities.
- **Drivers / adapters** — AI (`ai/*`), storage (`storage/*`), upload (tus), OTP, queue. Selected by environment, so development runs fully keyless.
- **Worker** (`src/worker`) — a BullMQ consumer process; in the default inline mode the pipeline runs in-process.
- **Realtime** — Socket.IO pushes pipeline status to the client.

3.2 Runtime topology

- **API** — Express 5 on Node $\geq$ 20, port 4000.
- **Database** — MongoDB 7 via Mongoose 9. Local dev can use an in-memory MongoDB (`mongodb-memory-server`) with `npm run dev:local`, requiring no Docker.
- **Redis 7 + BullMQ** — optional; when `REDIS_URL` is unset the queue driver falls back to `inline`.
- **Storage** — `local` filesystem or S3-compatible (`s3`, e.g. Cloudflare R2).
- **Frontend** — Vite 5 SPA served statically, installable as a PWA.

4 Contract-first design

`@sih/contracts` is the spine of the system.

4.1 Entities and schemas (Zod)

Every entity — Assessment, Declaration, QPMatch, AssessmentPlan, PublicTask, TaskResponse, Evidence, AIAnalysis, AssessorReview, Decision, CompetencyReport, Certificate, AuditLogEntry, WorkerProfile, User, TradePack — is a Zod schema with inferred TypeScript types. The same schemas validate backend input, type the frontend, and generate documentation. Zod 4 is used throughout.

4.2 The route table

The full HTTP surface is a single array of route descriptors (43 routes), each carrying method, path, tag, summary, auth role, params, query, body and response schema. Consequences:

- the backend *throws at boot* if any declared route lacks a handler (or a handler exists that is not declared) — the contract and the implementation cannot silently drift;
- the OpenAPI document is generated from the same table (`npm run openapi` writes the `openapi.json` document);
- auth middleware derives required roles from the table, not from scattered per-route decorations.

4.3 State machine as a contract

`ASSESSMENT_TRANSITIONS` is the authority for allowed status changes. The backend rejects illegal transitions; the frontend imports the same table (via `canTransition`) to render optimistic UI. The 12 statuses are shown in Appendix B.

5 Domain model and lifecycle

5.1 Roles

Four roles: **worker**, **assessor**, **authorized_assessor**, **admin**. A worker authenticates by phone OTP; assessors by email and password (bcrypt). Only an authorised assessor may record the final decision.

5.2 Entities

- **WorkerProfile** — demographics, trades, versioned for last-write-wins sync.
- **Declaration** — years of experience, work types, tools, tasks, sectors, employment types (free labels + free text).
- **QPMatch** — ranked NSQF qualification-pack candidates with a confidence and the signals that produced it.
- **AssessmentPlan** — an ordered list of *PublicTask*s with rubric weights summing to 100 %.
- **TaskResponse** — the worker's answer to one task; auto-scored where objective.
- **Evidence** — a media artefact (photo/video/audio/document) with a content hash, GPS, device and capture metadata.
- **AIAnalysis** — per-task AI result: score, confidence, routing and flags.
- **AssessorReview** — per-task final scores (accept or override), comments, outcome.
- **Decision** — APPROVED/REJECTED by an authorised assessor.
- **CompetencyReport** — NSQF result: overall score, NOS-wise results, recommendation, critical-safety pass, suggested NSQF level.
- **Certificate** — number, holder, QP, level, score, QR, and a revocation flag.
- **AuditLogEntry** — hash-chained record of every event.

5.3 Task types

Six task types drive the runner and the analysis adapters:

Type	Purpose / scoring
`mcq`	Multiple-choice knowledge check; deterministic auto-score.
`tool_identification`	Identify correct tools/instruments from an image set; deterministic.
`checklist`	Ordered/compound procedural checks; item-weighted.
`practical_photo`	Geotagged still evidence (min/max count) assessed by the vision adapter.
`practical_video`	Video evidence; keyframe + (optional) ASR analysis.
`narration`	Spoken explanation; ASR transcript confirmed by the worker, assessed by the LLM.

6 Assessment pipeline

6.1 Declaration and mapping

The worker submits a self-declaration. The mapping service combines deterministic keyword/NOS matching (using trade-pack aliases) with an optional embedding pass, and returns the top-3 QPs with confidence and signal explanations. The worker confirms a trade, moving the assessment to **MAPPED**.

6.2 Plan generation

For the chosen trade pack the planner instantiates tasks from *taskBlueprints* and MCQs from the pack's MCQ bank, attaches rubric weights (normalised to 100 %), evidence specifications (media

kinds, min/max counts, GPS and narration requirements) and NOS references. The plan is stored and the assessment becomes **PLAN_READY**.

6.3 Task runner and evidence capture

Answers are submitted per task and auto-scored where objective. Evidence is uploaded with *tus* (resumable, `@tus/server` + file store) and registered as metadata; the backend verifies the content hash on finalisation. GPS and a worker-in-frame cue support practical authenticity.

6.4 AI analysis

On `POST /submit` the assessment becomes **SUBMITTED** and the analysis pipeline is queued. The pipeline is modular:

- **Deterministic** scoring runs for objective items with no model call.
- **LLM** adapter scores narration and rubric judgements.
- **Vision** adapter assesses photos (tool identification, safety practice visible in frame).
- **ASR** adapter transcribes narration/audio; `ffmpeg` extracts video keyframes.

Each result carries a confidence; items below `AI_CONFIDENCE_HUMAN_ROUTE` (default 0.65) are routed to a human. Critical-safety items raise flags. When finished the assessment is **AI_ANALYZED**. Modes: `AI_MODE=mock` (keyless, deterministic — used in dev and tests) or `auto` with OpenAI-compatible endpoints.

6.5 Scoring engine

The score is computed by `scoring.service`:

- the **assessor's final score takes precedence** over the AI score (review-over-AI);
- per-task weighted aggregation to an overall score out of 100;
- **NOS-wise** roll-ups feed the competency report;
- a **critical-safety veto**: any unmet critical-safety competency fails the safety gate regardless of the average;
- NSQF level banding yields a suggested level;
- the recommendation is one of `certify`, `certify_with_bridge`, `reassess`, `not_certified`.

6.6 Review and decision

An assessor claims the case from the shared queue (**AI_ANALYZED** → **UNDER_REVIEW**); the queue is ordered oldest-first for fairness, and claiming is exclusive. The assessor reviews each task side by side with the deterministic and AI scores, flags and routing, accepting or overriding, then submits an outcome of **REVIEWED** or **REASSESS** (the latter returns the assessment to **IN_PROGRESS** for the round). An **authorized_assessor** records the final **APPROVED** or **REJECTED** decision; the competency report is generated and, on approval, a certificate is issued.

6.7 Report, certificate and verification

The competency report presents the overall score, NOS-wise results, consistency, assessor overrides, critical-safety status and suggested NSQF level. On approval a certificate is minted (`SIH-RPL-<year>-XXXXXXXX`) and rendered to PDF with `pdfkit`, with a QR code (`qrcode`) pointing at the public verifier `/verify/<certificateNo>`. The verifier is unauthenticated and returns validity, holder, trade, level, score and issue date. Revocation is supported via a flag on the certificate.

7 Offline-first and synchronisation

Worksites frequently lack connectivity, so the PWA is offline-capable:

- an **IndexedDB outbox** queues profile, declaration and answer operations; conflicting writes use last-write-wins with a client timestamp;
- every mutating request carries an **Idempotency-Key** so retries after reconnection cannot double-apply;
- a **sync push/pull** API reconciles the outbox with a cursor-based pull of server changes;
- uploads are **resumable** (tus), so an interrupted photo/video upload continues rather than restarting;
- a service worker precaches the app shell and applies stale-while-revalidate to contract GETs.

8 Security, privacy and integrity

- **Authentication** — short-lived JWT access tokens (default 30 min) with rotating refresh tokens (default 30 days, revocable); bcrypt for assessor passwords; phone OTP for workers with mock, Firebase or MSG91 providers.
- **Authorisation** — role checks derived from the route table; every assessment read is scoped to its owner or an assessor role.
- **Hardening** — Helmet, CORS allow-list, request size limits, and schema validation of all bodies/params/query.
- **Evidence integrity** — content is hashed (SHA-256); the audit log chains each entry to the previous hash, so tampering is detectable.
- **Privacy** — evidence is served only to authorised principals; fairness analytics are aggregate and descriptive (no attribute is ever used in an automated decision).

9 Fairness and multilingual support

9.1 Fairness

Regional language, gender and region attributes are recorded to *monitor* outcomes, not to drive them. `GET /api/v1/admin/fairness` reports approval rates and mean scores by dimension so that bias in AI-assisted scoring can be detected and corrected; the API and the UI both state explicitly that the analysis is descriptive only.

9.2 Multilingual

The UI, the assessment tasks and narration support English, Hindi and Marathi. Narration is captured as audio and transcribed; the worker confirms the transcript before it is scored, keeping the worker in control of what is assessed. Trade packs provide bilingual titles and aliases.

10 Frontend — DakshSetu PWA

10.1 Stack

React 18.3, React Router 6.28, TanStack Query 5.59 for server state, Vite 5.4 for build/HMR, and `vite-plugin-pwa` 0.20 (Workbox) for the manifest and service worker. The design system is a small set of accessible components (Card, Button, Field, Alert, StatusBadge, Stepper, . . .) using a government-friendly navy/soft palette.

10.2 Portals

- **Worker** — OTP login; dashboard of assessments; a declaration → map → plan wizard; the task runner (all six task types with evidence capture); the results screen with the NOS-wise report and certificate download.
- **Assessor** — queue; review screen with per-task AI vs deterministic scores, confidence, routing and safety flags; decision screen; fairness analytics.

- **Public** — the certificate verifier, reachable with no login.

10.3 Realtime and resilience

Socket.IO delivers pipeline status so the worker sees progress from submission through analysis. The app is installed like a native app, works offline, and resyncs automatically.

11 Testing and verification

11.1 Automated suite

The backend ships **124 tests across 14 suites** (Vitest + Supertest over an in-memory MongoDB, `AI_MODE=mock`, inline queue). Suites cover auth, assessments, declaration/mapping, planning, answers, evidence, analysis, scoring, review, report/decision/certificate, sync, fairness, trade packs and bilingual behaviour.

11.2 End-to-end walkthrough

The complete lifecycle was exercised through the running UI and API, not just unit tests. A worker was created by phone OTP, declared experience, was mapped to *Electrician - Domestic Solutions* (NSQF 3), received a 37-task plan with rubric weights summing to 100 %, completed all six task types, and submitted. Inline analysis produced **AI_ANALYZED**; an authorised assessor claimed the case, reviewed it and recorded a decision; the report listed NOS-wise results with a critical-safety veto; and the certificate's number (`SIH-RPL-2026-9614F4F9`) verified as valid on the public verifier, which returned the holder, trade, NSQF level and score. The audit trail showed the full hash-chained history from creation to approval.

11.3 Continuous checks

TypeScript is strict across all three workspaces; the frontend build (`tsc -noEmit && vite build`) and the backend type-check run in CI-style local commands. Because the contract throws at boot on a route/handler mismatch, an integration failure surfaces immediately.

12 Operations and deployment

12.1 Local development

- `npm run dev:local` — builds contracts, boots an in-memory MongoDB, and starts the API with keyless mock AI/OTP and the inline queue. No Docker required.
- `npm run infra:up` — Docker Compose with MongoDB 7 and Redis 7 (for the BullMQ driver).
- `npm run dev:frontend` — Vite dev server; `dev:worker` — the BullMQ worker.

12.2 Containers

The `Dockerfile` is a multi-stage `node:20-alpine` build: install, compile contracts then backend, and a runtime image that installs `ffmpeg` (for video) and runs as a non-root user with a `/health` healthcheck. `docker-compose.prod.yml` provides the production topology.

12.3 Configuration

A single `.env` drives all drivers: database, queue, storage, auth, OTP provider, and AI. Sensible defaults let the whole stack run keyless; real credentials switch individual adapters to Firebase/MSG91, OpenAI-compatible models, or S3/R2 storage without code changes. The public base URL and certificate prefix are configurable for QR links.

13 Trade pack catalogue

Four NSQF qualification packs are implemented as versioned JSON, each aligned to a real QP from a sector skill council, with NOS units, tools, safety topics, an MCQ bank, task blueprints, thresholds and bridge-training suggestions.

Pack	QP ID	Council	NSQF	NOS	MCQ	Blueprints
Electrician	PSS/Q6001	PSSC	3	11	28	11
Welder	CSC/Q0204	CGSC	3	5	24	10
Plumber	PSC/Q0104	WMPSC	4	8	24	12
Mason	CON/Q0103	CSDCI	3	8	24	12

Electrician - Domestic Solutions (PSS/Q6001, PSSC, NSQF 3) is the flagship pack: house wiring, appliances, protective devices and safety, with 22 tools, 16 safety topics and 5 sources of truth.

14 API surface

The API exposes 43 routes grouped by tag: `meta` (health), `auth`, `profile`, `assessments`, `declaration`, `mapping`, `plan`, `evidence`, `analysis`, `review`, `report`, `sync`, `trades` and `admin`. Worker routes require a worker token; assessor routes an assessor role; the decision route specifically requires `authorized_assessor`; verification and trade discovery are public. The complete route reference is in Appendix A and is generated from `contracts/openapi.json`.

15 Limitations and future work

- The AI adapters are keyless mocks in development; production accuracy depends on a configured model, and prompts/calibration should be validated per trade.
- Video analysis uses keyframes and optional ASR rather than continuous pose/object tracking.
- Certificates are PDF + QR with a server-side verifier; verifiable credentials (W3C VC) and a blockchain anchor are natural extensions.
- Offline conflict resolution is last-write-wins; per-field merge with a CRDT for concurrent multi-device edits is future work.
- Larger, psychometrically validated item banks and calibration of the confidence threshold are required before national rollout.

A API route reference

Method	Path	Auth	Summary
GET	/health	public	Liveness + dependency checks
POST	/api/v1/auth/otp/request	public	Request phone OTP
POST	/api/v1/auth/otp/verify	public	Verify OTP, get session
POST	/api/v1/auth/assessor/register	public	Register assessor
POST	/api/v1/auth/assessor/login	public	Assessor login
POST	/api/v1/auth/refresh	public	Refresh tokens
POST	/api/v1/auth/logout	any	Revoke refresh token
GET	/api/v1/auth/me	any	Current user + profile
GET	/api/v1/profile	worker	Get own worker profile
PATCH	/api/v1/profile	worker	Update profile (LWW + version)
POST	/api/v1/assessments	worker	Create assessment (DRAFT)
GET	/api/v1/assessments	any	List assessments (role-scoped)
GET	/api/v1/assessments/:id	any	Get assessment + history
POST	/api/v1/assessments/:id/declaration	worker	Self-declaration (DECLARED)
GET	/api/v1/assessments/:id/declaration	any	Get declaration
POST	/api/v1/assessments/:id/map	worker	AI NSQF mapping (MAPPED)
GET	/api/v1/assessments/:id/mapping	any	Get QP match
POST	/api/v1/assessments/:id/plan	worker	Generate plan (PLAN_READY)
GET	/api/v1/assessments/:id/plan	any	Get plan (keys stripped)
POST	/api/v1/assessments/:id/start	worker	Start / resume (IN_PROGRESS)
POST	/api/v1/assessments/:id/answers	worker	Submit task answers
GET	/api/v1/assessments/:id/answers	any	List task responses
POST	/api/v1/assessments/:id/submit	worker	Submit (queues AI)
POST	/api/v1/assessments/:id/evidence	worker	Register evidence metadata
GET	/api/v1/assessments/:id/evidence	any	List evidence
GET	/api/v1/evidence/:id/content	any	Stream evidence bytes
GET	/api/v1/assessments/:id/analysis	any	Get AI analysis
GET	/api/v1/assessments/:id/analysis/status	any	Pipeline status
POST	/api/v1/assessments/:id/review/claim	assessor	Claim review
POST	/api/v1/assessments/:id/review	assessor	Submit review
GET	/api/v1/assessments/:id/review	any	Get latest review
GET	/api/v1/assessor/queue	assessor	Assessor queue
GET	/api/v1/assessments/:id/report	any	Competency report
POST	/api/v1/assessments/:id/decision	authorized_-assessor	Final decision
GET	/api/v1/assessments/:id/decision	any	Get decision
GET	/api/v1/certificates/:id/pdf	any	Certificate PDF (QR)
GET	/api/v1/assessments/:id/audit	assessor	Tamper-evident audit trail
GET	/api/v1/verify/:certificateNo	public	Public certificate verification
POST	/api/v1/sync/push	worker	Push offline ops (idempotent)
GET	/api/v1/sync/pull	worker	Pull changes since cursor
GET	/api/v1/trades	public	List trade packs
GET	/api/v1/trades/:id	public	Full trade pack (QP/NOS/rubric)
GET	/api/v1/admin/fairness	assessor	Fairness stats by dimension

B State machine and task types

B.1 Statuses

Status	Meaning
DRAFT	Assessment created, nothing declared.
DECLARED	Self-declaration submitted.
MAPPED	A qualification pack has been confirmed.
PLAN_READY	Personalised task plan generated.
IN_PROGRESS	Worker is answering tasks / capturing evidence.
SUBMITTED	Answers submitted; AI pipeline queued.
AI_ANALYZED	Automated analysis complete.
UNDER_REVIEW	An assessor has claimed the case.
REASSESS	Assessor returned the case for another round.
REVIEWED	Review complete; awaiting final decision.
APPROVED	Authorised assessor approved; certificate issued.
REJECTED	Authorised assessor rejected.

B.2 Allowed transitions

From	Allowed to
DRAFT	DECLARED
DECLARED	MAPPED
MAPPED	PLAN_READY
PLAN_READY	IN_PROGRESS
IN_PROGRESS	SUBMITTED
SUBMITTED	AI_ANALYZED
AI_ANALYZED	UNDER_REVIEW
UNDER_REVIEW	REVIEWED, REASSESS
REASSESS	IN_PROGRESS
REVIEWED	APPROVED, REJECTED
APPROVED	(terminal)
REJECTED	(terminal)